

5-1 · 분산 처리와 Spark 아키텍처 · 실습 1H

Spark UI로 잡 구조 읽기

Job → Stage → Task 와 DAG, 셔플이 Stage 를 가른다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

- ① Jobs 탭에서 action 마다 Job 한 줄이 생긴 것을 본다
- ② groupBy 로 Stage 가 2개가 되는 것과 explain 의 Exchange 를 찾는다
- ③ Stages 탭에서 Task 수 = 파티션 수를 확인한다
- ④ cache 전후의 실행 시간을 직접 쟀다

Executors 탭에 driver 하나뿐인 화면을 기억해 두세요 — 5-2 에서 워커로 채워집니다

여섯 단계 — 탭을 하나씩 열며 계층을 읽는다

앞 실습의 셀과 localhost:4040 을 그대로 씁니다 — 상단 탭 Jobs · Stages · Storage · Environment · Executors

1. Jobs 탭	2. 셔플 만들기	3. Stages 탭	4.5. DAG · 실행자	6. cache
action 마다 한 줄	groupBy Stage 2개	Task 수 = 파티션 수	박스과 선 driver 하나	count 세 번 시간 비교

Jobs 탭 — action 마다 한 줄

열 · 항목	뜻
한 줄	지금까지 실행한 action 하나
Duration	그 Job 이 걸린 시간
Stages	그 Job 이 몇 단계로 나뉘었는지
Job 클릭	Stage 목록 + DAG Visualization

확인: 앞 실습의 count() 와 show(5) 가 한 줄씩 · 둘 다 Stage 1개 — 셔플이 없었기 때문

groupBy 로 셔플을 만들어 Stage 를 늘린다

코드 — 같은 키를 한곳에

같은 키를 모아야 하므로 노드 간 재배치 = 셔플

UI 와 explain()

Job 을 클릭하면 Stage 가 2개 · 계획에 Exchange

VS

range

1 ~ 100만

withColumn

bucket = id % 10

groupBy·count

show()

- Stage 0 — 읽고 나누는 단계
- Stage 1 — 모아서 세는 단계
- 그 사이가 셔플
- Exchange hashpartitioning(...) = 셔플

Stages 탭에서 Task 를 본다

1 Stages 탭 → 방금 실행한 Stage 클릭
Tasks 표에 Task 하나당 한 줄 — 파티션 하나가 Task 하나

2 Duration 이 Task 마다 다르다
한 Task 만 유난히 오래 걸리면 데이터가 한쪽에 몰린 것(스큐)

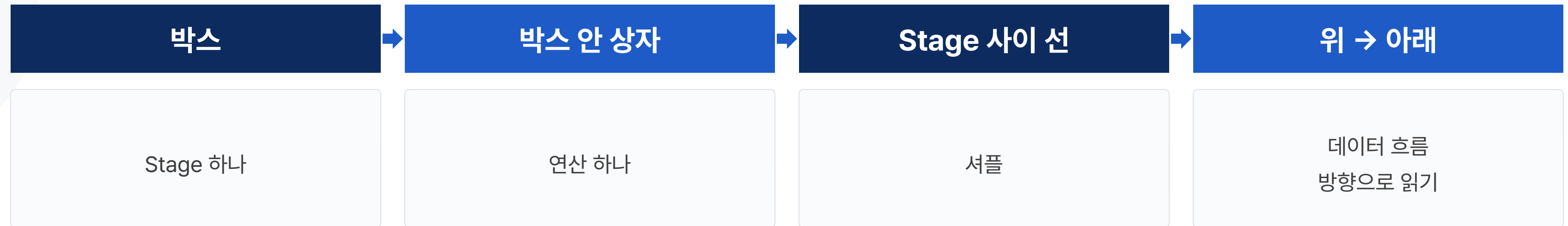
3 Summary Metrics 의 Min · Median · Max
비교하면 치우침이 한눈에 보인다

4 Stage 1 의 Task 수는 200 인 경우가 많다
기본 셔플 파티션 수 — 데이터가 작는데 200개면 과하다, 5-5 에서 조절

확인할 것

**Task 수 =
파티션 수**

DAG 를 읽는다 — 박스가 Stage, 선이 셔플



groupBy 잡: 박스 두 개, 사이에 선 하나 — 이론 챕터의 도해와 같은 구조

5단계 Executors 탭: driver 하나만 — local 모드는 드라이버가 익스큐터 역할까지 겸한다 · Cores · Memory · Task Time 열을 봐 두기

cache 의 효과를 재 본다 — count 세 번

big	spark.range(1, 20_000_001).filter("id % 7 = 0")
첫 번째	t0 = time.time(); big.count(); t1 = time.time()
두 번째(캐시 없음)	big.count() 다시 — 비슷하게 걸린다
캐시 채우기	big.cache(); big.count()
세 번째(캐시 있음)	big.count() — 눈에 띄게 빠르다
해제	big.unpersist()

확인: 세 번째가 눈에 띄게 빠르다 · Storage 탭에 캐시된 데이터가 나타난다 · 캐시는 메모리를 차지하니 정말 여러 번 쓸 때만

일부러 실패시키기

셀을 나가면 4040 도 사라진다

cache 까지 끝낸 뒤 일부러 셀을 나가 봅니다 — UI 가 언제까지 사는지 겪어 봅니다

exit()	4040 새로그침	이유	다시 띄우면	끝난 잡을 보려면
셀을 빠져나간다	열리지 않는다	앱이 살아 있는 동안만 UI	새 UI 가 생긴다	History Server (심화)

이 1시간이 끝날 때 확인할 것

- 1 Jobs 탭에 `count()` 와 `show(5)` 가 한 줄씩, 둘 다 Stage 1개
- 2 `groupBy` 잡은 Stage 2개 · `explain()` 에 Exchange 가 보인다
- 3 Stage 의 Task 수가 앞 실습의 파티션 수와 같다
- 4 DAG 에 박스 두 개, 사이에 선 하나
- 5 Executors 탭에 driver 하나뿐
- 6 캐시 뒤 세 번째 `count` 가 빠르고 Storage 탭에 나타난다

5-1 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

오늘 본 개념 아홉 개를 표로 정리하고, 5-2 클러스터를 위한 이미지·메모리를 확인합니다

✓ 개념 9개 — 한 줄 정의와 UI 위치

✓ docker images | grep spark

✓ Docker 메모리 4GB 이상(8GB 권장)

✓ 자주 겪는 오류 네 가지

가져갈 것

cache 전후 시간 세 개 메모